

ONE PAGE SUMMARY

ฟีดแบคที่ได้รับและสิ่งที่ทีมเรียนรู้

Pack_Glow Beta | Sprint Review | 6 พฤษภาคม 2568

ฟีดแบคที่ได้รับ

สิ่งที่ทีมเรียนรู้และปรับปรุง

1	 ฟีดแบคที่ได้รับ Function Point Report ยังไม่ครบถ้วน PM ยังไม่ได้จัดทำ Function Point Report ให้ครบ และไม่มี comment ประกอบ ทำให้ไม่สามารถประเมินขอบเขตงานได้	 สิ่งที่ทีมเรียนรู้และปรับปรุง PM ต้องรับผิดชอบจัดทำและอัปเดต Function Point Report อย่างต่อเนื่องตลอด Sprint ไม่ใช่ทำเฉพาะตอนส่งงาน เพราะเอกสารนี้สะท้อนขอบเขตและความคืบหน้าของโปรเจกต์โดยตรง
2	 ฟีดแบคที่ได้รับ จำนวน Commit บน GitHub น้อยเกินไป Dev ฝ่าย Frontend มี commit น้อยกว่าเกณฑ์ ทำให้ติดตาม progress การทำงานได้ยาก และดูเหมือนมีส่วนร่วมน้อย	 สิ่งที่ทีมเรียนรู้และปรับปรุง ทีมควรแยก commit ให้ย่อยและสม่ำเสมอตลอดการทำงาน เพราะ commit history คือหลักฐานแสดงกระบวนการทำงาน ไม่ใช่แค่ผลลัพธ์สุดท้าย
3	 ฟีดแบคที่ได้รับ Code Review บน PR ขาด Comment QA approve PR โดยไม่มี review comment ใดๆ ผู้สอนระบุว่า การ review ต้องมีความเห็นเป็นลายลักษณ์อักษรเสมอ	 สิ่งที่ทีมเรียนรู้และปรับปรุง การ review ที่ดีต้องระบุว่าตรวจสอบอะไร พบอะไร และตัดสินใจอย่างไร การกด approve โดยไม่มี comment ทำให้ไม่มีร่องรอยของกระบวนการ QA ที่ตรวจสอบได้
4	 ฟีดแบคที่ได้รับ ตั้งค่า CI/CD ช้าเกินไป DevOps เพิ่งมาตั้งค่า CD ใน Sprint 3 ทำให้เกิดปัญหาตอน deploy และต้องแก้ช้าหลายรอบ	 สิ่งที่ทีมเรียนรู้และปรับปรุง CI/CD ควรตั้งค่าตั้งแต่ Sprint แรกพร้อมกับโครงสร้างโปรเจกต์ การตั้งค่าทำให้ปัญหาสะสมและต้องแก้พร้อมกันทีเดียว ซึ่งเสียเวลาและสร้างความเสี่ยงโดยไม่จำเป็น
5	 ฟีดแบคที่ได้รับ Scrum Master ขาด Contribution บน GitHub Scrum Master แทบไม่มี activity บน GitHub เลย ขณะที่ทีมอื่น Scrum Master มีส่วนร่วมทำ document และ pull request ด้วย	 สิ่งที่ทีมเรียนรู้และปรับปรุง Scrum Master ต้องมี artifact ที่จับต้องได้ เช่น Sprint notes, Retrospective summary หรือ User story ที่ช่วย refine เพื่อให้กระบวนการ facilitate ตรวจสอบได้บนระบบ
6	 ฟีดแบคที่ได้รับ Reflection ยังไม่ลึกพอ คำตอบใน Retrospective ยังอยู่ในระดับผิวเผิน ไม่ได้สะท้อนมาจากประสบการณ์จริงที่ลึกพอ	 สิ่งที่ทีมเรียนรู้และปรับปรุง Reflection ที่มีคุณภาพต้องถามตัวเองว่า ทำไมถึงนั้นถึงเกิดขึ้น มีสัญญาณเตือนอะไรที่มองข้าม และถ้าย้อนกลับไปได้จะตัดสินใจต่างอย่างไร — ไม่ใช่แค่บอกว่า 'อยากทำให้ดีขึ้น'

บทสรุปสำหรับทีม

ทีมมีพื้นฐานที่ดี — รู้หน้าที่ ทำงานได้ลึก และผลลัพธ์น่าประทับใจ สิ่งที่ต้องพัฒนาต่อคือความสม่ำเสมอในกระบวนการ ทั้งการ document งาน การ review อย่างมีคุณภาพ และการสะท้อนคิดที่ลึกจากประสบการณ์จริง

คำตอบเพิ่มเติมของคำถามในส่วนของทีม

Q1: ทำไม CI/CD ล้มเหลว 3 ครั้งจาก 5 run ล่าสุด และทำไม failure สะสม

CI/CD ที่ล้มเหลวเกิดจากปัญหา configuration หลัก ๆ 3 จุด

ครั้งแรกเกิดจากใน workflow ตั้งค่า Python เป็น 3.14 ซึ่ง GitHub Actions ยังไม่รองรับ ทำให้ backend setup ไม่ผ่านตั้งแต่ขั้นแรก

ครั้งที่สองเกิดจาก Dockerfile ของ frontend สั่งรัน server.js แต่หลัง build Next.js แล้วไม่มีไฟล์อยู่ใน container ทำให้ frontend start ไม่ได้

ครั้งที่สามเกิดจาก E2E test พยายามเรียก localhost:3000 แต่ใน CI ไม่มี backend service รันอยู่ และ port ที่ใช้ใน Docker Compose กับ CI ไม่ตรงกัน ทำให้ test เชื่อมต่อ API ไม่ได้

สาเหตุที่ failure สะสม เพราะทีมยัง push เข้า main โดยตรง ไม่มี branch protection บังคับให้ CI ผ่านก่อน และช่วงนั้นกำลังรีบแก้ deployment เลยแก้ทีละจุดแล้ว push ต่อ ทำให้เกิด failed runs หลายครั้งติดกัน บทเรียนคือควรมี staging branch และให้ CI ผ่านก่อน merge เข้า main

Q2: ทำไมเลือก Stack ชับซ้อน และ SQLite เหมาะไหม

ทีมเลือก stack นี้เอง เพราะอยากให้ Pack&Glow เป็น full-stack project จริง มี frontend, backend, database, deployment และ AI/API integration ครบ ไม่ได้ตั้งใจเลือกให้ซับซ้อนเกิน แต่พอทำจริงพบว่า Next.js + FastAPI + Docker + AI APIs มีรายละเอียดเยอะ โดยเฉพาะตอน deploy

SQLite +aiosqlite ถูกเลือกเพราะโปรเจกต์นี้เป็น MVP/demo สำหรับรายวิชา คาดว่าผู้ใช้พร้อมกันไม่เยอะ และ setup ง่ายกว่า PostgreSQL มาก ทำให้ทีมมีเวลาไปโฟกัส feature หลัก เช่น trip, checklist, outfit และ AI assistant

ทีมรู้ว่า SQLite มีข้อจำกัดเรื่อง concurrent writes เลยมีการลดความเสี่ยงบางส่วน เช่น ตั้ง timeout=30 และใช้ WAL mode แต่ถ้ามาระบบจะใช้งานจริงหรือมีผู้ใช้เยอะ ควรเปลี่ยนไปใช้ PostgreSQL เพราะรองรับ concurrent users ได้ดีกว่า

Q3: จัดการ API keys, rate limit และ cost ยังไง

API keys ของ OpenRouter และ OpenWeatherMap ไม่ได้ใส่ไว้ตรง ๆ ใน source code แต่เก็บผ่าน GitHub Secrets และ inject เข้า backend .env ตอน deploy แล้วให้ FastAPI อ่านค่าผ่าน settings

API key owner คือ Sukit Inpim ซึ่งเป็นคนดูแลฝั่ง deployment และ external services หลัก

สิ่งที่ทำไว้แล้วคือ ถ้าไม่มี API key หรือ API ใช้ไม่ได้ ระบบจะ fallback ไปใช้ rules-based response แทน ทำให้ระบบยังพอทำงานได้โดยไม่ล้มทั้งหมด

เรื่อง cost ทีมตั้ง limit ใน OpenRouter ไว้ที่ \$1 ต่อวัน เพื่อกันค่าใช้จ่ายเกิน ส่วน OpenWeatherMap ใช้ free tier ประมาณ 1,000 calls/day ซึ่งเพียงพอสำหรับ demo

ข้อที่ยังต้องปรับปรุงคือ ยังไม่มีระบบ track cost ราย user หรือราย feature ชัดเจน และ SlowAPI ที่ใช้เป็นแค่ rate limit endpoint ยังไม่ได้คุ้มต้นทุน AI call โดยตรง

Q4: Dependabot PRs ค้าง 8 รายการ จะ merge หรือ ignore

Dependabot PRs ที่ค้างไม่ได้ตั้งใจ ignore แต่ทีมเลือกเลื่อนไว้ก่อน เพราะช่วงท้ายโปรเจกต์กำลังแก้ bug, test และ deploy ถ้า merge dependency updates หลายตัวพร้อมกัน อาจทำให้เกิดปัญหาใหม่และ debug ยากกว่าเดิม

กลยุทธ์ที่ควรทำคือ merge เฉพาะตัวที่เป็น security patch ก่อน ส่วน patch/minor update ให้ merge ทีละ PR หลังจาก test ผ่าน และถ้าเป็น major update หรือเกี่ยวกับ framework หลัก เช่น Next.js, React, Tailwind หรือ database library ควรเลื่อนไปหลังส่งงาน

สรุปคือทีมเลือก freeze dependency ชั่วคราวเพื่อให้ระบบนิ่งก่อน แต่หลังส่งงานควรกลับมา review Dependabot PRs ทีละตัว ไม่ควรปล่อยค้างยาว ๆ